

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Computer Networks

Data Privacy and Security

Data Science and Management (**DASMA**)

LUISS



Computer Networks

Understanding computer networks is essential in today's interconnected world:

- **Communication:** How devices exchange data across the internet
- **Protocol Stack:** The layered architecture that makes networking possible
- **TCP/IP:** The fundamental protocols powering the internet
- **Applications:** From web browsing to video streaming
- **Security:** Understanding vulnerabilities and protection mechanisms

We'll explore the **TCP/IP stack** and see how your computer communicates with the world!

What is a Computer Network?

A **computer network** is a collection of interconnected devices that can exchange data and share resources.

Key Components:

- **Nodes:** Devices connected to the network (computers, phones, servers, routers)
- **Links:** Physical or wireless connections between nodes
- **Protocols:** Rules governing how data is transmitted
- **Data:** Information being exchanged between nodes

What is a Computer Network?

A **computer network** is a collection of interconnected devices that can exchange data and share resources.

Key Components:

- **Nodes:** Devices connected to the network (computers, phones, servers, routers)
- **Links:** Physical or wireless connections between nodes
- **Protocols:** Rules governing how data is transmitted
- **Data:** Information being exchanged between nodes

EXAMPLE: When you browse a website, your computer (node) connects through WiFi (link) using HTTP protocol to fetch data from a web server (another node).

Types of Networks

Networks can be classified by their **scale** and **coverage**:

Types of Networks

Networks can be classified by their **scale** and **coverage**:

- Local Area Network (**LAN**)
 - *Coverage*: Small geographic area (home, office, building)
 - *Examples*: Your home WiFi, office network, the LUISS campus network
 - *Characteristics*: Typically 1 Gbps (Ethernet) or less (WiFi), private ownership

Types of Networks

Networks can be classified by their **scale** and **coverage**:

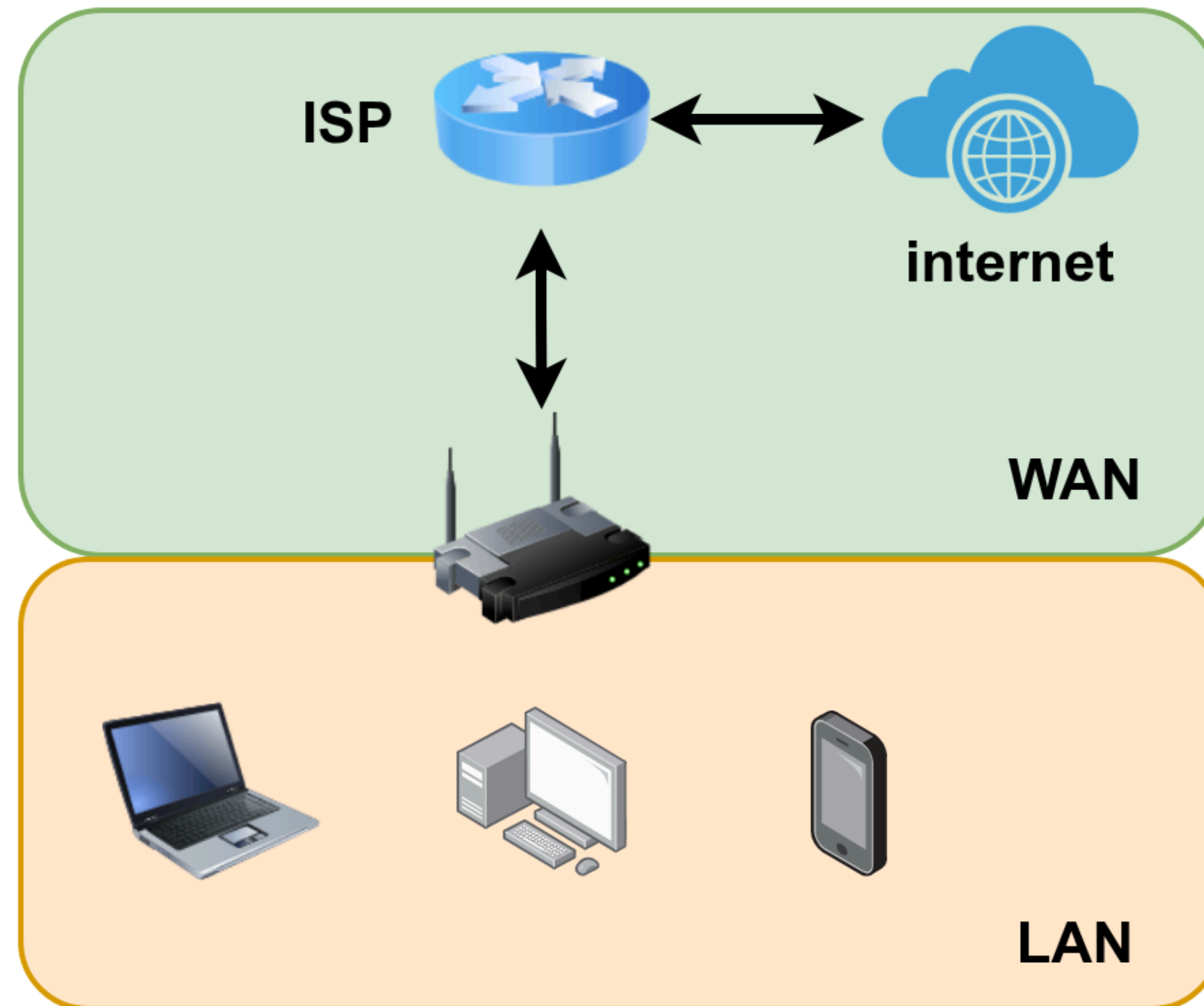
- Local Area Network (**LAN**)
 - *Coverage*: Small geographic area (home, office, building)
 - *Examples*: Your home WiFi, office network, the LUISS campus network
 - *Characteristics*: Typically 1 Gbps (Ethernet) or less (WiFi), private ownership
- Wide Area Network (**WAN**)
 - *Coverage*: Large geographic area (cities, countries, continents)
 - *Examples*: The Internet, corporate networks across multiple cities
 - *Characteristics*: High-speed backbone links (10–100 Gbps), uses public infrastructure

Types of Networks

Networks can be classified by their **scale** and **coverage**:

- Local Area Network (**LAN**)
 - *Coverage*: Small geographic area (home, office, building)
 - *Examples*: Your home WiFi, office network, the LUISS campus network
 - *Characteristics*: Typically 1 Gbps (Ethernet) or less (WiFi), private ownership
- Wide Area Network (**WAN**)
 - *Coverage*: Large geographic area (cities, countries, continents)
 - *Examples*: The Internet, corporate networks across multiple cities
 - *Characteristics*: High-speed backbone links (10–100 Gbps), uses public infrastructure
- Other Types:
 - Metropolitan Area Network (**MAN**): City-wide coverage
 - Personal Area Network (**PAN**): Very small area (Bluetooth devices)

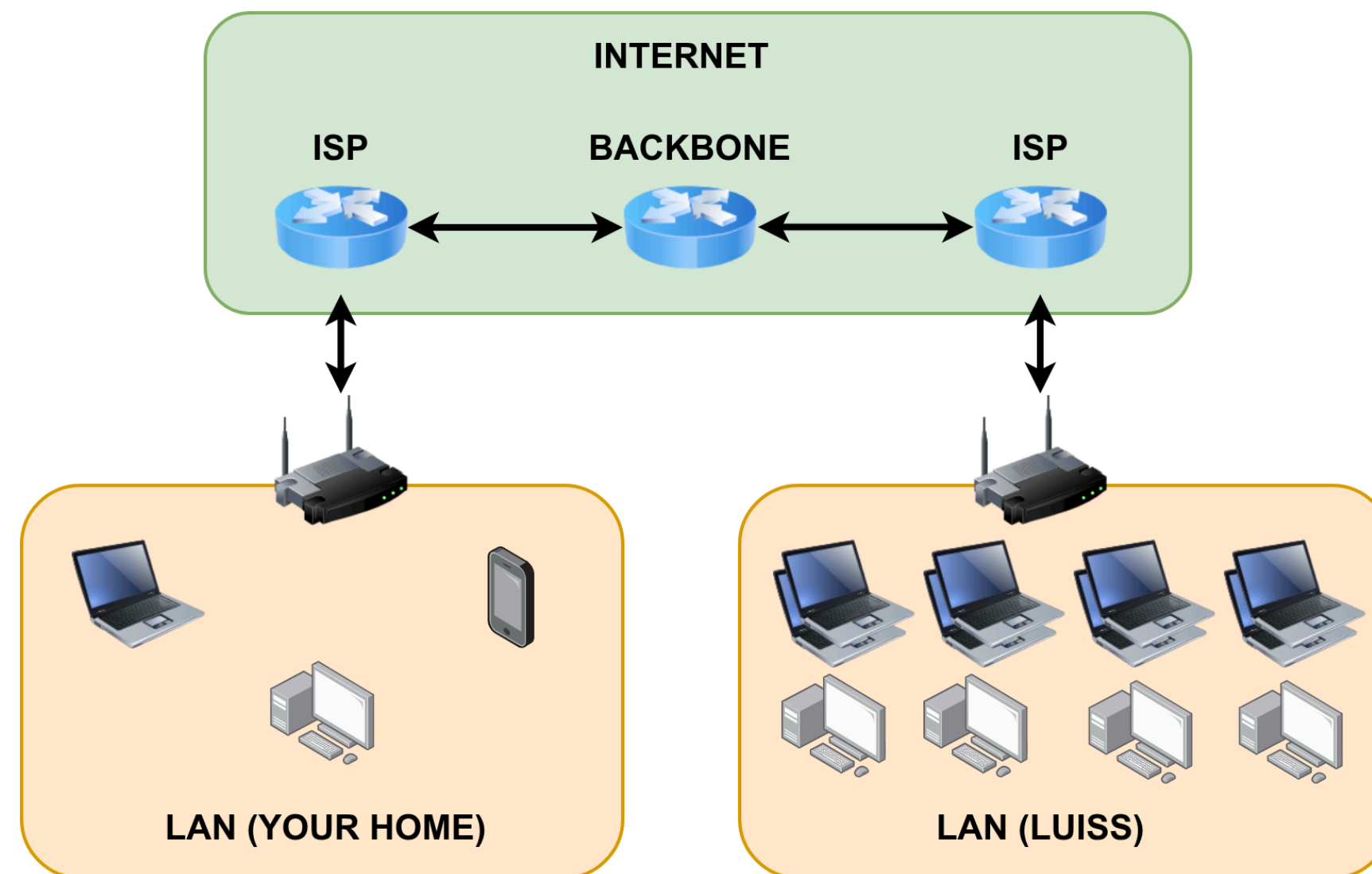
Types of Networks



The Internet: A Network of Networks

The **Internet** is the largest WAN, connecting billions of devices worldwide.

Internet Service Provider (ISP): Companies that provide internet access and maintain the infrastructure connecting networks together.



Why Do We Need Network Protocols?

Imagine two philosophers, one in Greece and one in China, who want to exchange ideas. They don't speak the same language, they live thousands of kilometers apart, and they have no direct way to communicate.

Why Do We Need Network Protocols?

Imagine two philosophers, one in Greece and one in China, who want to exchange ideas. They don't speak the same language, they live thousands of kilometers apart, and they have no direct way to communicate.

They need to agree on:

- A **common language** (e.g., use a translator)
- A **means of delivery** (e.g., letters via postal service)
- A **format** (e.g., write on paper, use envelopes with addresses)
- **Rules for the conversation** (e.g., wait for a reply before sending the next message)

Each of these is a **layer** of rules – a **protocol** – that makes communication possible.

Why Do We Need Network Protocols?

Imagine two philosophers, one in Greece and one in China, who want to exchange ideas. They don't speak the same language, they live thousands of kilometers apart, and they have no direct way to communicate.

They need to agree on:

- A **common language** (e.g., use a translator)
- A **means of delivery** (e.g., letters via postal service)
- A **format** (e.g., write on paper, use envelopes with addresses)
- **Rules for the conversation** (e.g., wait for a reply before sending the next message)

Each of these is a **layer** of rules – a **protocol** – that makes communication possible.

Computer networks face the **same challenges**: devices must agree on shared rules (**protocols**) to communicate reliably across different technologies and distances.

The OSI Model: A Conceptual Framework

The **OSI (Open Systems Interconnection)** model is a conceptual framework with **7 layers**:

Layer	Name	Function	Examples
7	Application	User-facing applications	HTTP, FTP, SMTP, DNS
6	Presentation	Data format translation, encryption	SSL/TLS, JPEG, MPEG
5	Session	Session management	NetBIOS, RPC
4	Transport	End-to-end communication	TCP, UDP
3	Network	Routing and addressing	IP, ICMP
2	Data Link	Physical addressing, error detection	Ethernet, WiFi, ARP
1	Physical	Raw bit transmission	Cables, fiber, radio waves

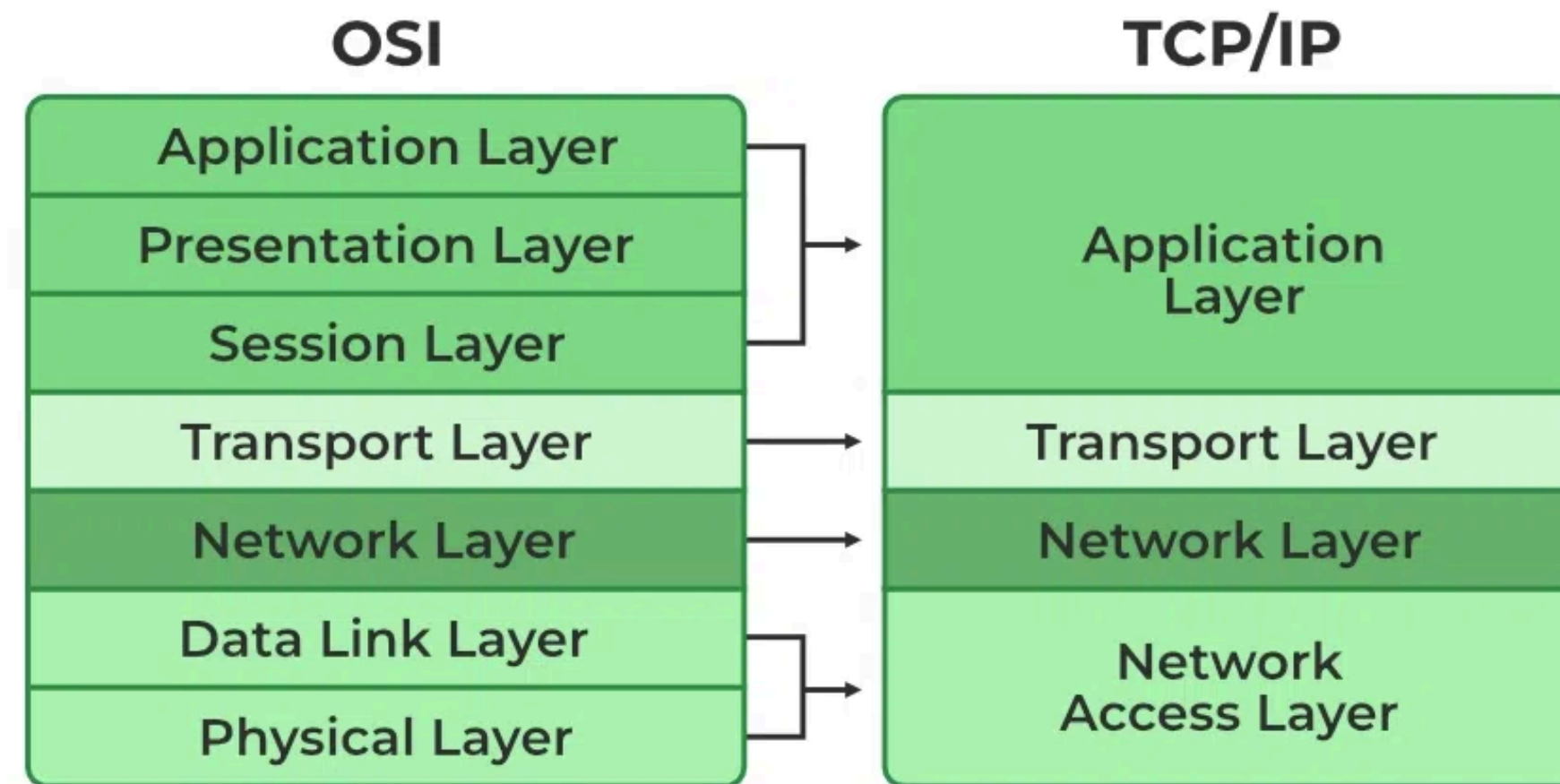
The OSI Model: A Conceptual Framework

The **OSI (Open Systems Interconnection)** model is a conceptual framework with **7 layers**:

Layer	Name	Function	Examples
7	Application	User-facing applications	HTTP, FTP, SMTP, DNS
6	Presentation	Data format translation, encryption	SSL/TLS, JPEG, MPEG
5	Session	Session management	NetBIOS, RPC
4	Transport	End-to-end communication	TCP, UDP
3	Network	Routing and addressing	IP, ICMP
2	Data Link	Physical addressing, error detection	Ethernet, WiFi, ARP
1	Physical	Raw bit transmission	Cables, fiber, radio waves

NOTE: The OSI model is **theoretical**. In practice, we use the **TCP/IP model**, simpler and more practical.

OSI to TCP/IP



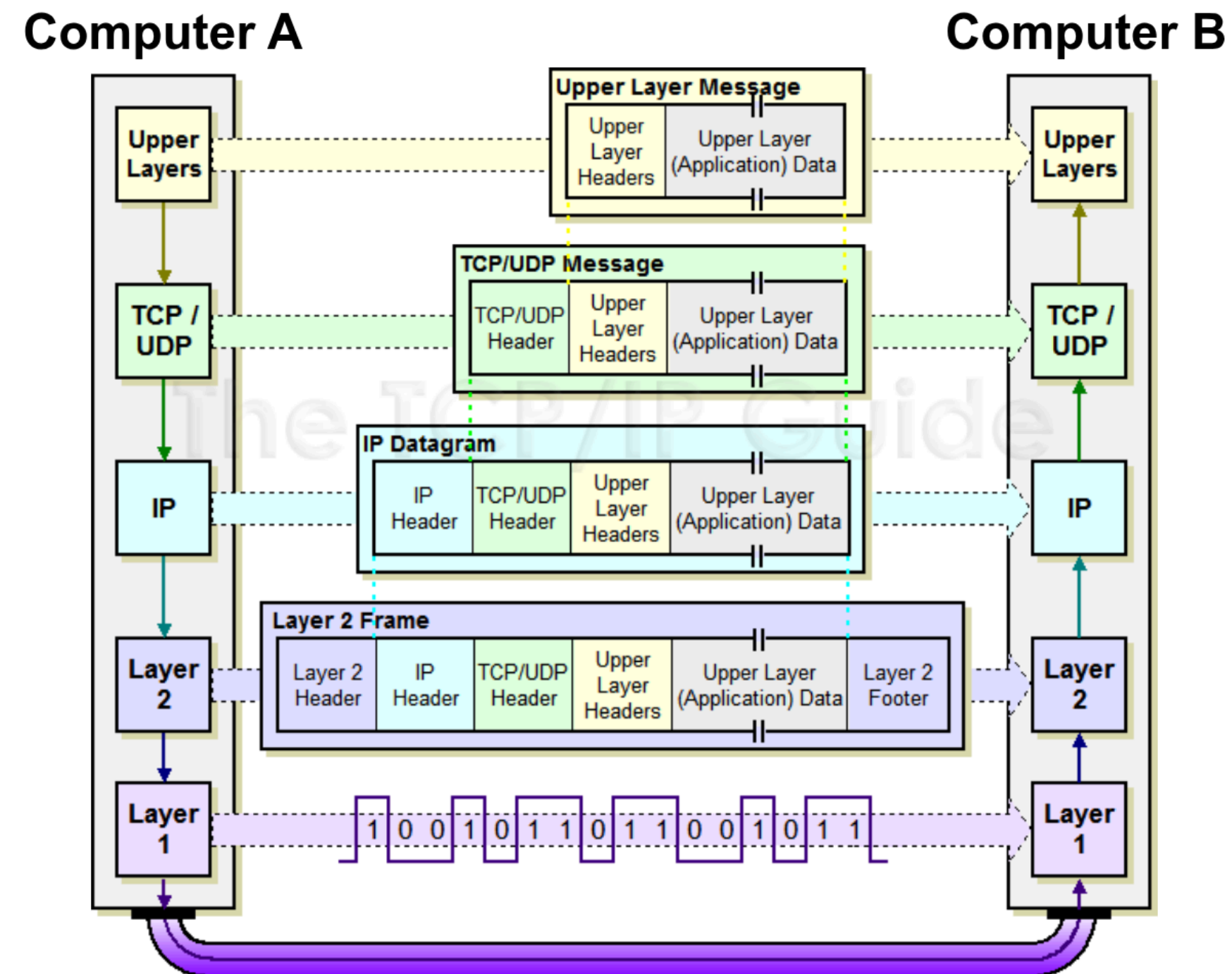
The TCP/IP Model: The Internet's Foundation

The **TCP/IP model** has **4 layers** and is the actual protocol suite used on the internet:

Layer	OSI Equivalent	Function	Key Protocols
Application	5-7	User applications and services	HTTP, HTTPS, FTP, SMTP, DNS, SSH
Transport	4	End-to-end data delivery	TCP, UDP
Internet	3	Routing across networks	IP, ICMP
Network Access	1-2	Physical transmission	Ethernet, WiFi, PPP, ARP

Each layer adds its own **header** to the data, creating a **protocol stack**.

The TCP/IP Model



Goals of each layer

- **Network Access Layer:** transfer data between devices on the same physical network
 - two devices connected to the same WiFi or Ethernet can communicate directly
- **Internet Layer:** route packets across multiple networks
 - transfer data from your home network to a web server across the internet
- **Transport Layer:** provide reliable or unreliable delivery of data between applications
 - allows multiple applications on the same device to communicate over the network
- **Application Layer:** enable user-level applications to communicate over the network
 - specific protocols for web browsing, email, file transfer, etc.

Layer 1: Network Access



Layer 1: Network Access Layer

The **Network Access Layer** handles physical transmission of data:

- Converting digital data to electrical/optical/radio signals
- Physical addressing (MAC addresses)
- Accessing the physical medium (cables, wireless)
- Error detection at the hardware level
- Nodes are identified by their **MAC addresses**

Layer 1: Network Access Layer

The **Network Access Layer** handles physical transmission of data:

- Converting digital data to electrical/optical/radio signals
- Physical addressing (MAC addresses)
- Accessing the physical medium (cables, wireless)
- Error detection at the hardware level
- Nodes are identified by their **MAC addresses**

MAC Address: A unique 48-bit identifier assigned to work interface card (NIC). Used for *local* communication.

EXAMPLE: 00:1A:2B:3C:4D:5E.

Comparison of Physical Network Links

Technology	Type	Max Speed	Max Distance	Pros	Cons	Use Cases
Fiber Optic	Wired (Light)	100+ Gbps	100+ km	Very fast, immune to EMI, secure	Expensive, fragile	Backbone, data centers
Copper (Ethernet)	Wired (Electrical)	10 Gbps	100m	Cost-effective, easy setup	Limited distance, EMI-prone	LANs, home networks
WiFi (802.11)	Wireless (Radio)	1-10 Gbps	50-100m	No cables, mobile	Interference, security risks	Home, offices, hotspots
4G/5G	Wireless (Cellular)	100 Mbps - 10 Gbps	Several km	Wide coverage, mobile	Expensive, variable speed	Mobile, remote areas, IoT

Physical Network Links

ADSL

≈7mm



ETHERNET

≈11mm

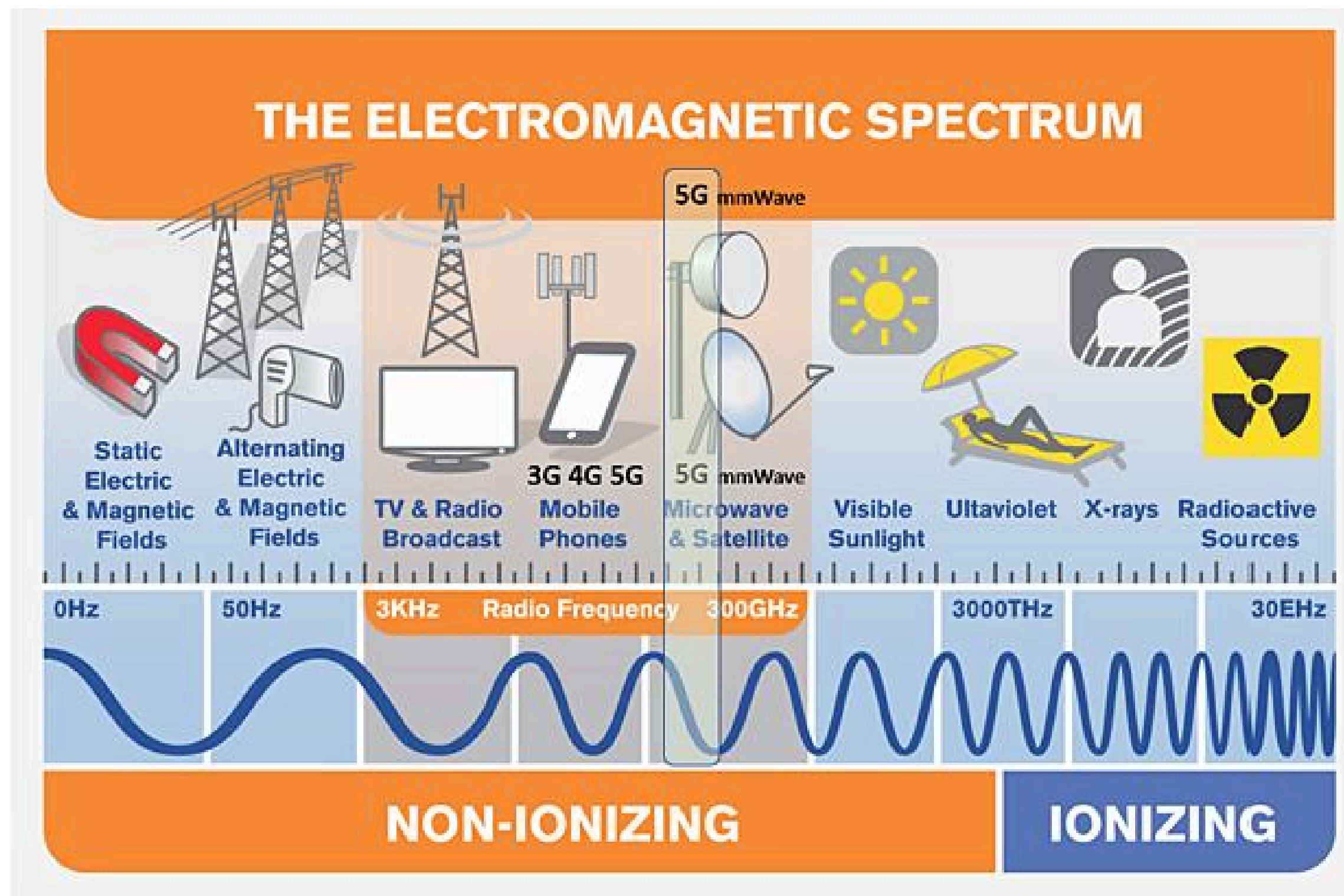


OPTIC FIBER

≈3mm



Electromagnetic Spectrum



WiFi Channels

Comparison of Wi-Fi Standards

	 Wi-Fi 5	 Wi-Fi 6	 Wi-Fi 6E	 Wi-Fi 7
Standard	IEEE 802.11ac	IEEE 802.11ax	IEEE 802.11ax	IEEE 802.be*
Transmit Frequency	5 GHz only	2.4 and 5 GHz	2.4, 5, and 6 GHz	2.4, 5, and 6 GHz
Max Number of Streams	8	8	8	16
Max Data Rate per Stream	866 Mb/s	1.2 Gb/s	1.2 Gb/s	2.9 Gb/s
Max Theoretical Data Rate	6.93 Gb/s	9.61 Gb/s	9.61 Gb/s	46.1 Gb/s
Typical Speed	1.3 Gb/s	5 Gb/s	5 Gb/s	18 Gb/s (preliminary)

*In development

Layer 2: Internet



Layer 2: Internet Layer

The **Internet Layer** is responsible for routing data across networks:

- Routes packets from source to destination
- Works across different types of networks
- Provides logical addressing (IP addresses)
- Key Protocol: **Internet Protocol (IP)**
- Nodes are identified by their **IP addresses**

Layer 2: Internet Layer

The **Internet Layer** is responsible for routing data across networks:

- Routes packets from source to destination
- Works across different types of networks
- Provides logical addressing (IP addresses)
- Key Protocol: **Internet Protocol (IP)**
- Nodes are identified by their **IP addresses**

We are using IPv4 but (slowly) moving to IPv6: extremely costly to change entire infrastructure!

IP Addressing

IP Address: A unique numerical identifier for each device on a network

IP Addressing

IP Address: A unique numerical identifier for each device on a network

- **IPv4:** 32-bit address (e.g., 192.168.1.1)
 - Format: Four numbers (0-255) separated by dots
 - Problem: Only 4.3 billion addresses (running out!)

IP Addressing

IP Address: A unique numerical identifier for each device on a network

- **IPv4:** 32-bit address (e.g., 192.168.1.1)
 - Format: Four numbers (0-255) separated by dots
 - Problem: Only 4.3 billion addresses (running out!)
- **IPv6:** 128-bit address (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334)
 - Format: Eight groups of hexadecimal numbers
 - Solution: 340 undecillion addresses (virtually unlimited)

IP Addressing: Public vs Private

Public IP Addresses:

- Globally unique and routable on the internet
- Assigned by ISPs
- Required for direct internet communication
- Example: 8.8.8.8 (Google's DNS server)

Private IP Addresses:

- Used within local networks (LANs)
- Not routable on the internet
- Can be reused in different networks
- Ranges:
 - 10.0.0.0 to 10.255.255.255
 - 172.16.0.0 to 172.31.255.255
 - 192.168.0.0 to 192.168.255.255

IP Addressing: Public vs Private

Public IP Addresses:

- Globally unique and routable on the internet
- Assigned by ISPs
- Required for direct internet communication
- Example: 8.8.8.8 (Google's DNS server)

Private IP Addresses:

- Used within local networks (LANs)
- Not routable on the internet
- Can be reused in different networks
- Ranges:
 - 10.0.0.0 to 10.255.255.255
 - 172.16.0.0 to 172.31.255.255
 - 192.168.0.0 to 192.168.255.255

To allow multiple devices with private IPs to share a single public IP address when accessing the internet, we use **NAT (Network Address Translation)** (see later).

What is my IP in LUISS? [Ethernet]



Enter Keywords or IP Address...

Search

[ABOUT](#) [PRESS](#) [PODCAST](#) [SUPPORT](#)

MY IP

IP LOOKUP

HIDE MY IP

VPNS

TOOLS

LEARN

My IP Address is:

IPv4: ? **193.204.156.25**

IPv6: ? **Not detected**

My IP Information:

ISP: Consortium GARR

City: Rome

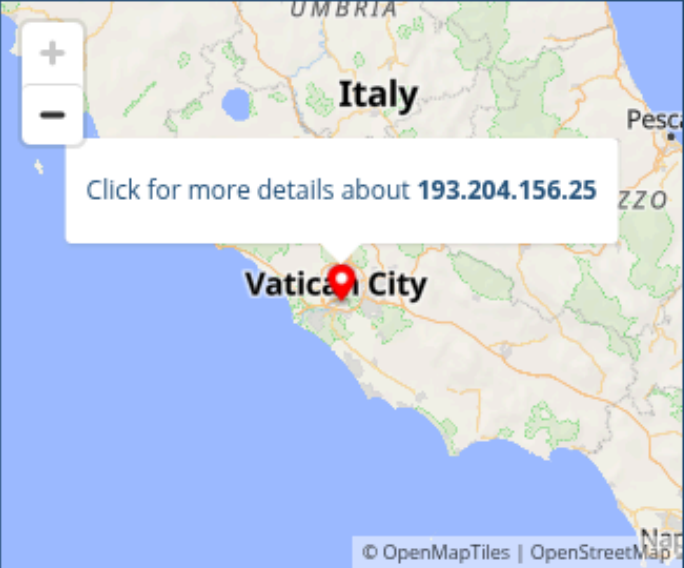
Region: Lazio

Country: Italy

Your location may be exposed!

HIDE MY IP ADDRESS NOW

[Show Complete IP Details](#)



Click for more details about 193.204.156.25

Location not accurate?

[Update My IP Location](#)

LUISS



23

What is my IP in LUISS? [LUISSWpa]



Enter Keywords or IP Address...

Search

ABOUTPRESSPODCASTSUPPORT

MY IP

IP LOOKUP

HIDE MY IP

VPNS

TOOLS

LEARN

My IP Address is:

IPv4: 193.204.156.30

IPv6: Checking...

My IP Information:

ISP: Consortium GARR

City: Rome

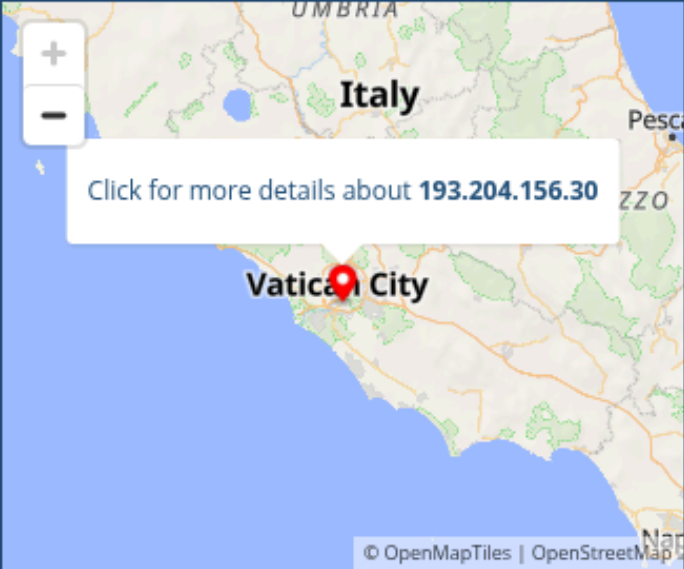
Region: Lazio

Country: Italy

Your location may be exposed!

HIDE MY IP ADDRESS NOW

Show Complete IP Details



Location not accurate?

Update My IP Location

What is my IP in LUISS? [Eduroam]

151.14.193.221   Star

 Rome, Lazio, IT 

- Summary >
- Geolocation
- Device info
- Privacy
- ASN
- Company
- Abuse
- Q&A

Need more data or want to access it via API or data downloads? Sign up to get free access [Sign up for free >](#)

Summary	
ASN	AS1267 - WIND TRE S.P.A.
Hostname	No Hostname
Range	151.14.0.0/16
Company	LUISS GUIDO CARLI
Hosted domains	0
Privacy	 False
Anycast	 False
ASN type	ISP
Abuse contact	abuse@wind.it

Public IP ranges

Universities use the GARR network:

- Sapienza Università di Roma: from 151.100.0.0 to 151.100.255.255
- LUISS Guido Carli: 193.204.154.0 - 193.204.154.255 (also other ranges)

Some IP ranges used by italian ISPs:

- Vodafone Italia S.p.A. (AS30722): 5.88.0.0-5.95.255.255
- Fastweb SpA (AS12874): 93.32.0.0-93.47.255.255
- WIND TRE S.P.A. (AS1267): 151.48.0.0-151.63.255.255
- Telecom Italia S.p.A. (AS16232): 2.192.0.0-2.199.255.255

Subnet Mask

A **subnet mask** defines which portion of an IP address identifies the **network** and which portion identifies the **host** (device).

- Written in the same format as an IP address (4 octets, each 0–255)
- Each octet is a sequence of 8 bits: 255 = 11111111, 0 = 00000000
- Bits set to 1 → **network** part, bits set to 0 → **host** part
- The 1s are always **contiguous** from the left (no mixing allowed)

Subnet Mask: Bit-Level View

EXAMPLE: IP 192.168.1.10 with subnet mask 255.255.255.0

	Decimal	Binary
IP Address	192.168.1.10	11000000.10101000.00000001.00001010
Subnet Mask	255.255.255.0	11111111.11111111.11111111.00000000

Mask bits = 1 → **network** portion, Mask bits = 0 → **host** portion:

	Bits 1–8	Bits 9–16	Bits 17–24	Bits 25–32
IP (binary)	11000000	10101000	00000001	00001010
Mask (binary)	11111111	11111111	11111111	00000000
Role	Network	Network	Network	Host

- **Network part:** 192.168.1 → identifies the network
- **Host part:** .10 → identifies the device within that network

Common subnet mask octets and their binary representation:

Decimal	Binary	Network bits	Host bits
255	11111111	8	0
128	10000000	1	7
192	11000000	2	6
224	11100000	3	5
240	11110000	4	4
248	11111000	5	3
252	11111100	6	2
254	11111110	7	1
0	00000000	0	8

EXAMPLE: IP 192.168.1.10 with subnet mask 255.255.255.0:

	Octet 1	Octet 2	Octet 3	Octet 4
IP Address	192	168	1	10
Subnet Mask	255	255	255	0
Network Part	192	168	1	—
Host Part	—	—	—	10

Every subnet has **two reserved addresses** that cannot be assigned to any device:

- **Network address** (192.168.1.0) — all host bits are 0. It identifies the network itself and is never assigned to a device.
- **Broadcast address** (192.168.1.255) — all host bits are 1. A packet sent to this address is delivered to **every device** on the network simultaneously (e.g., used by DHCP when a device joins and shouts "Who can give me an IP?").

Because these two addresses are reserved, the number of **usable host addresses** is always:

$$\text{usable hosts} = 2^{\text{host bits}} - 2$$

For a /24 network (8 host bits): $2^8 - 2 = 256 - 2 = \mathbf{254}$ usable hosts.

EXAMPLE: IP 192.168.1.10 with subnet mask 255.255.255.0:

	Octet 1	Octet 2	Octet 3	Octet 4
IP Address	192	168	1	10
Subnet Mask	255	255	255	0
Network Part	192	168	1	—
Host Part	—	—	—	10

Every subnet has **two reserved addresses** that cannot be assigned to any device:

- **Network address** (192.168.1.0) — all host bits are 0. It identifies the network itself and is never assigned to a device.
- **Broadcast address** (192.168.1.255) — all host bits are 1. A packet sent to this address is delivered to **every device** on the network simultaneously (e.g., used by DHCP when a device joins and shouts "Who can give me an IP?").

Because these two addresses are reserved, the number of **usable host addresses** is always:

$$\text{usable hosts} = 2^{\text{host bits}} - 2$$

For a /24 network (8 host bits): $2^8 - 2 = 256 - 2 = \mathbf{254}$ usable hosts.

NOTE: Two devices can communicate **directly** (without a router) only if they are on the **same network**, i.e., they share the same network part of their IP address.

CIDR Notation

CIDR (Classless Inter-Domain Routing) is a compact way to express an IP address and its subnet mask:

- Format: IP_address/prefix_length
- The **prefix length** is the number of bits set to 1 in the subnet mask
- Replaces the old "classful" system (Class A, B, C) with flexible subnetting

EXAMPLE: Converting subnet mask to CIDR prefix:

Subnet Mask	Binary	CIDR
255.0.0.0	11111111.00000000.00000000.00000000	/8
255.255.0.0	11111111.11111111.00000000.00000000	/16
255.255.255.0	11111111.11111111.11111111.00000000	/24
255.255.255.128	11111111.11111111.11111111.10000000	/25
255.255.255.192	11111111.11111111.11111111.11000000	/26

CIDR Examples

192.168.1.0/24 – A typical home network:

- Subnet mask: 255.255.255.0
- Network: 192.168.1.0
- Host range: 192.168.1.1 – 192.168.1.254
- Usable hosts: **254**

10.0.0.0/8 – A large corporate network:

- Subnet mask: 255.0.0.0
- Network: 10.0.0.0
- Host range: 10.0.0.1 – 10.255.255.254
- Usable hosts: **16,777,214**

192.168.1.0/26 – A small office subnet:

- Subnet mask: 255.255.255.192
- /26 means 26 network bits → **6 host bits** → $2^6 = 64$ total addresses
- Subtract 2 reserved addresses (network + broadcast): $64 - 2 = \mathbf{62}$ usable hosts
- Network address: 192.168.1.0 (host bits all 0)
- Broadcast address: 192.168.1.63 (host bits all 1)
- Host range: 192.168.1.1 – 192.168.1.62
- Usable hosts: **62**

CIDR: Are Two Devices on the Same Network?

To check if two IPs are on the same network, apply the subnet mask (bitwise AND) to both:

EXAMPLE: Subnet 192.168.1.0/24 (mask 255.255.255.0)

	Device A	Device B	Device C
IP	192.168.1.10	192.168.1.200	192.168.2.5
IP AND Mask	192.168.1.0	192.168.1.0	192.168.2.0
Network	192.168.1.0	192.168.1.0	192.168.2.0

- A and B → **same network** ✓ (can communicate directly)
- A and C → **different networks** ✗ (need a router)

CIDR: Another Example

EXAMPLE: Subnet 172.16.0.0/16 (mask 255.255.0.0)

	Device A	Device B	Device C
IP	172.16.5.10	172.16.200.1	172.17.0.1
IP AND Mask	172.16.0.0	172.16.0.0	172.17.0.0
Network	172.16.0.0	172.16.0.0	172.17.0.0

- A and B → **same network ✓** (both in 172.16.x.x)
- A and C → **different networks ✗** (172.16.x.x vs 172.17.x.x)

Gateway (Router)

A **gateway** is a device (typically a **router**) that connects two or more different networks and forwards packets between them.

- A router has multiple network interfaces, each connected to a **different network** (LAN)
- When a device wants to send data to an IP that is **not on its local network**, it forwards the packet to the gateway
- The gateway examines the destination IP and forwards the packet toward the correct network
- This process repeats hop-by-hop until the packet reaches its destination

Gateway (Router)

A **gateway** is a device (typically a **router**) that connects two or more different networks and forwards packets between them.

- A router has multiple network interfaces, each connected to a **different network** (LAN)
- When a device wants to send data to an IP that is **not on its local network**, it forwards the packet to the gateway
- The gateway examines the destination IP and forwards the packet toward the correct network
- This process repeats hop-by-hop until the packet reaches its destination

EXAMPLE: Device 192.168.1.10/24 wants to reach 8.8.8.8 (Google DNS):

1. Device checks: is 8.8.8.8 on my network (192.168.1.0/24)? **No**
2. Device sends the packet to its **gateway** (e.g., 192.168.1.1)
3. The gateway (router) forwards the packet toward 8.8.8.8 across the internet

The gateway configured on a device is called the **default gateway** – it is the "exit door" from the local network.

DHCP: Dynamic Host Configuration Protocol

DHCP automatically assigns IP addresses to devices on a network:

1. Device connects to network and broadcasts: "I need an IP address!"
2. DHCP server responds with available IP address and configuration
3. Device accepts and uses the assigned IP address for a lease period
4. When lease expires, device must renew or get a new address

DHCP: Dynamic Host Configuration Protocol

DHCP automatically assigns IP addresses to devices on a network:

1. Device connects to network and broadcasts: "I need an IP address!"
2. DHCP server responds with available IP address and configuration
3. Device accepts and uses the assigned IP address for a lease period
4. When lease expires, device must renew or get a new address

What DHCP Provides:

- IP address: the unique identifier for the device
- Subnet mask: defines the network segment
- Default gateway (router): the path to external networks
- DNS servers: for domain name resolution (see later)
- Lease duration (typically 24 hours to 7 days)

Other Internet Layer Protocols

- **ICMP (Internet Control Message Protocol):** Error reporting and diagnostics
 - Used by tools like ping and traceroute
 - Reports unreachable hosts, network congestion, timeouts
 - Example: "Destination Host Unreachable" error messages

Other Internet Layer Protocols

- **ICMP (Internet Control Message Protocol):** Error reporting and diagnostics
 - Used by tools like ping and traceroute
 - Reports unreachable hosts, network congestion, timeouts
 - Example: "Destination Host Unreachable" error messages
- **ARP (Address Resolution Protocol):** Maps IP addresses to MAC addresses
 - Operates between Internet Layer and Network Access Layer
 - When a device knows an IP address but needs the MAC address
 - Example: Router needs to know MAC address of 192.168.1.10 to deliver packet on local network

Layer 3: Transport



Layer 3: Transport Layer

Transport Layer provides end-to-end communication between applications through two main protocols:

Layer 3: Transport Layer

Transport Layer provides end-to-end communication between applications through two main protocols:

- **TCP (Transmission Control Protocol):**

- Connection-oriented: Establishes connection before sending data
- Reliable: Guarantees delivery and error-free transmission
- Packets may arrive out of order, but TCP **reorders** them before delivering to the application
- Flow control: Manages data rate to prevent overwhelming receiver
- Use cases: Web browsing, email, file transfer (when reliability matters)

- **UDP (User Datagram Protocol):**

- Connectionless: No connection establishment
- Unreliable: No delivery guarantees, packets may be lost or arrive out of order
- Fast: Lower overhead, minimal delay
- Use cases: Video streaming, online gaming, DNS queries

Layer 3: Transport Layer

Transport Layer provides end-to-end communication between applications through two main protocols:

- **TCP (Transmission Control Protocol):**

- Connection-oriented: Establishes connection before sending data
- Reliable: Guarantees delivery and error-free transmission
- Packets may arrive out of order, but TCP **reorders** them before delivering to the application
- Flow control: Manages data rate to prevent overwhelming receiver
- Use cases: Web browsing, email, file transfer (when reliability matters)

- **UDP (User Datagram Protocol):**

- Connectionless: No connection establishment
- Unreliable: No delivery guarantees, packets may be lost or arrive out of order
- Fast: Lower overhead, minimal delay
- Use cases: Video streaming, online gaming, DNS queries

TCP is more reliable but slower, while UDP is faster but less reliable. The choice depends on application requirements.

TCP: The 3-Way Handshake

Before any data is sent, TCP establishes a reliable connection using a **3-way handshake**:

1. **SYN** – Client sends a synchronization request to the server
2. **SYN-ACK** – Server acknowledges and sends its own synchronization request
3. **ACK** – Client acknowledges the server's request

TCP: The 3-Way Handshake

Before any data is sent, TCP establishes a reliable connection using a **3-way handshake**:

1. **SYN** – Client sends a synchronization request to the server
2. **SYN-ACK** – Server acknowledges and sends its own synchronization request
3. **ACK** – Client acknowledges the server's request

After the handshake, a **reliable bidirectional channel** is established. Data can now flow in both directions with guaranteed delivery and ordering.

TCP: The 3-Way Handshake

Before any data is sent, TCP establishes a reliable connection using a **3-way handshake**:

1. **SYN** – Client sends a synchronization request to the server
2. **SYN-ACK** – Server acknowledges and sends its own synchronization request
3. **ACK** – Client acknowledges the server's request

After the handshake, a **reliable bidirectional channel** is established. Data can now flow in both directions with guaranteed delivery and ordering.

To close the connection, a similar process occurs with **FIN** (finish) and **ACK** packets. This ensures both sides agree the communication is complete.

Moving from devices to applications

PROBLEM: Multiple applications can run on the same device simultaneously (web browser, email client, video call, etc.), each needing its own communication channel. How does the device know which incoming packet belongs to which application?

Moving from devices to applications

PROBLEM: Multiple applications can run on the same device simultaneously (web browser, email client, video call, etc.), each needing its own communication channel. How does the device know which incoming packet belongs to which application?

SOLUTION: Ports

Moving from devices to applications

PROBLEM: Multiple applications can run on the same device simultaneously (web browser, email client, video call, etc.), each needing its own communication channel. How does the device know which incoming packet belongs to which application?

SOLUTION: Ports

A **port** is a numerical identifier (0–65535) that identifies a specific application or service on a device.

- It is the **address at the Transport Layer** – just as IP addresses identify devices, ports identify applications on a device
- Each application listens on a specific port (e.g., a web server on port 80)
- When a packet arrives, the Transport Layer uses the port number to deliver it to the correct application

Ports: Addressing Applications

Port numbers identify specific applications or services on a device:

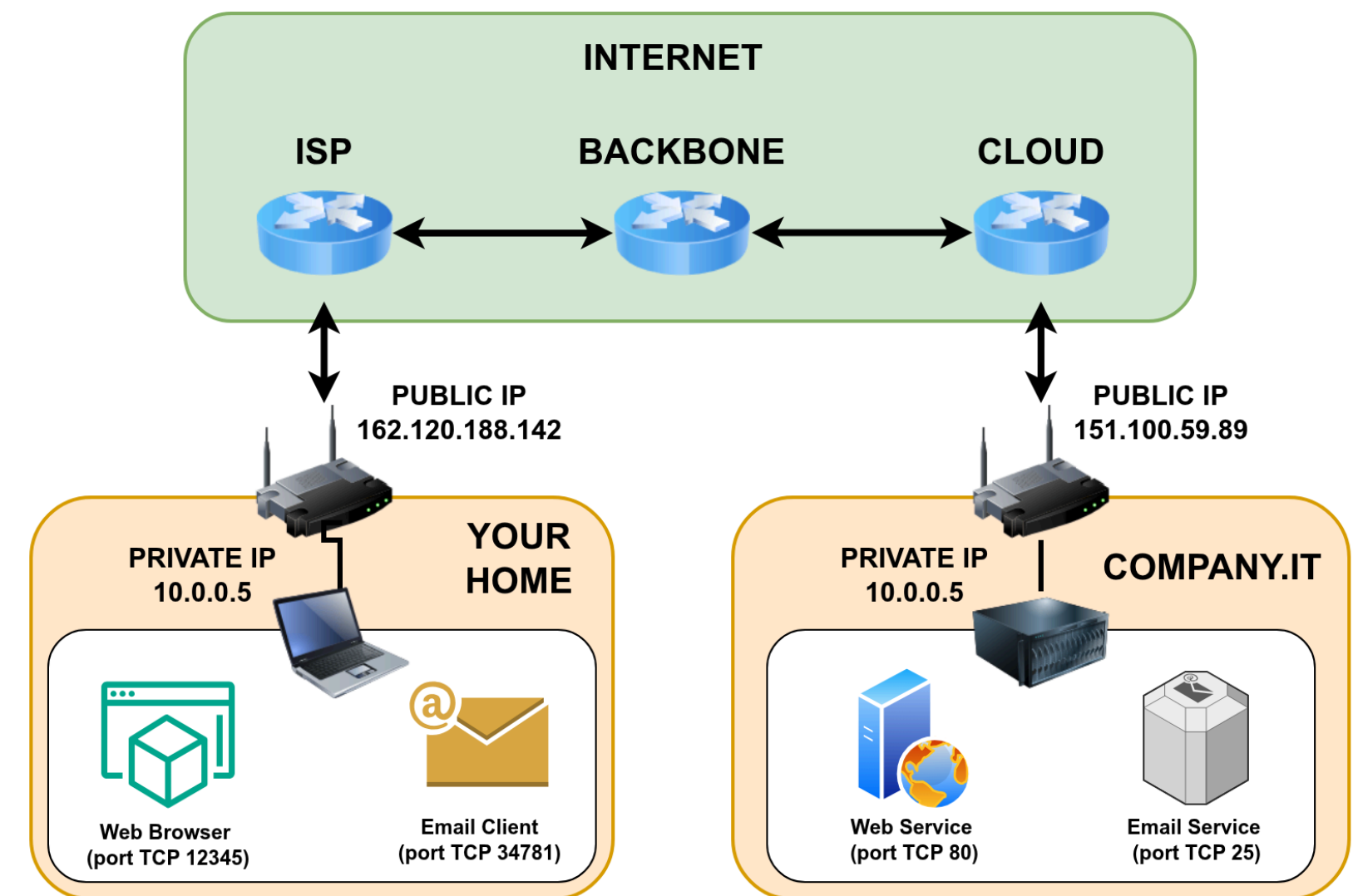
- Range: 0–65535

Port Categories:

Range	Category	Usage
0–1023	Well-known ports	Standard services
1024–49151	Registered ports	Application-specific
49152–65535	Dynamic/Private ports	Temporary client connections

Common Well-Known Ports

- **80**: HTTP (web browsing)
- **443**: HTTPS (secure web browsing)
- **22**: SSH (secure remote access)
- **21**: FTP (file transfer)
- **25**: SMTP (email sending)
- **53**: DNS (domain name resolution)
- **3389**: RDP (Remote Desktop Protocol)



WHEN VISITING www.company.it:

SRC=162.120.188.142:12345 DST=151.100.59.89:80

WHEN SENDING EMAIL FROM user@company.it: SRC=162.120.188.142:34781 DST=151.100.59.89:25

NAT (Network Address Translation) and Ports

NAT allows multiple devices with private IPs to share a single public IP address when accessing the internet. It exploits ports to differentiate between multiple internal devices.

NAT (Network Address Translation) and Ports

NAT allows multiple devices with private IPs to share a single public IP address when accessing the internet. It exploits ports to differentiate between multiple internal devices.

When a device sends a request to the internet:

1. The NAT device replaces the private IP address with its own public IP address.
2. It assigns a unique port number to the request.
3. The NAT device keeps track of the mapping between the internal IP address/port and the external IP address/port.

NAT (Network Address Translation) and Ports

NAT allows multiple devices with private IPs to share a single public IP address when accessing the internet. It exploits ports to differentiate between multiple internal devices.

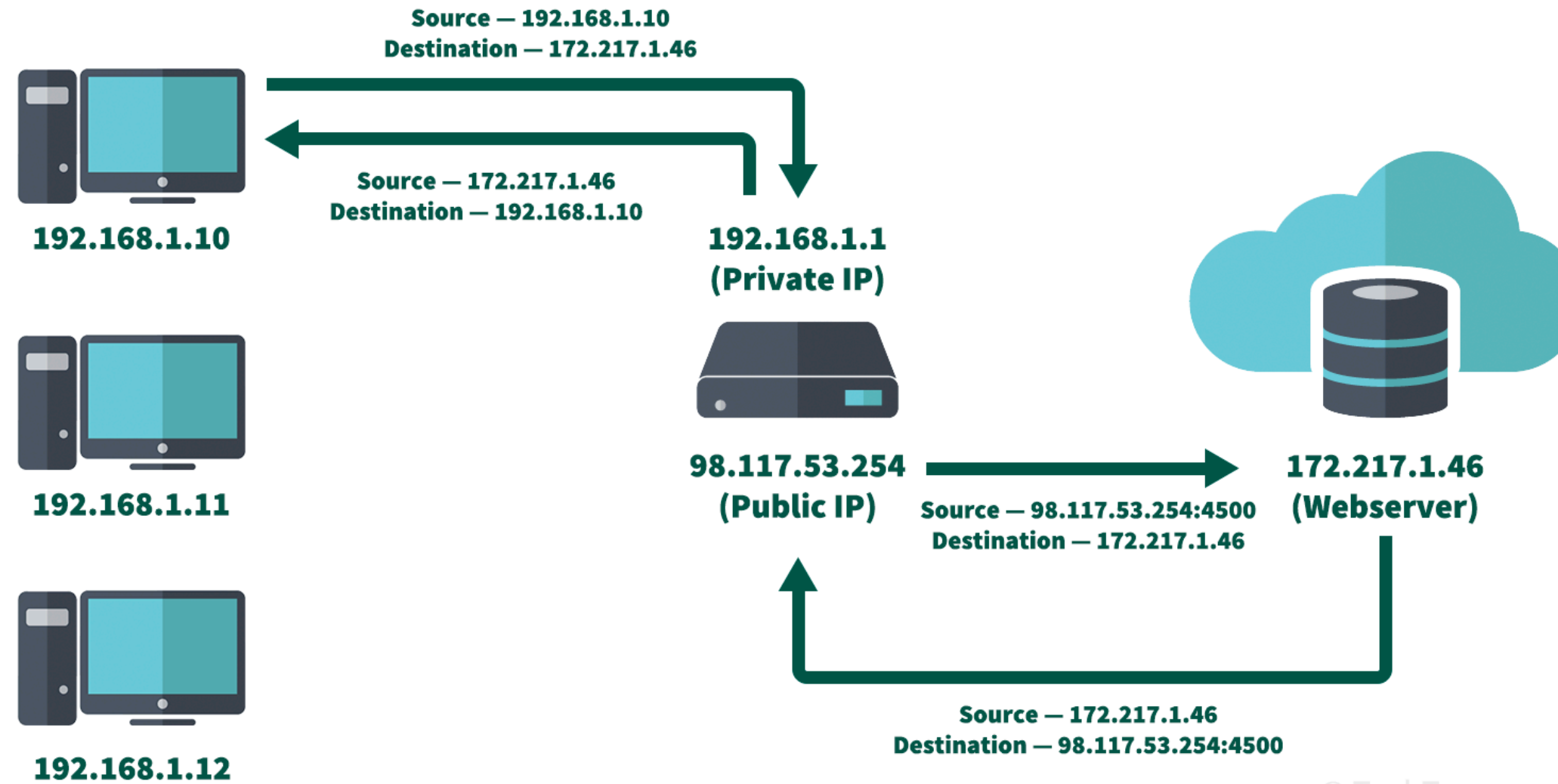
When a device sends a request to the internet:

1. The NAT device replaces the private IP address with its own public IP address.
2. It assigns a unique port number to the request.
3. The NAT device keeps track of the mapping between the internal IP address/port and the external IP address/port.

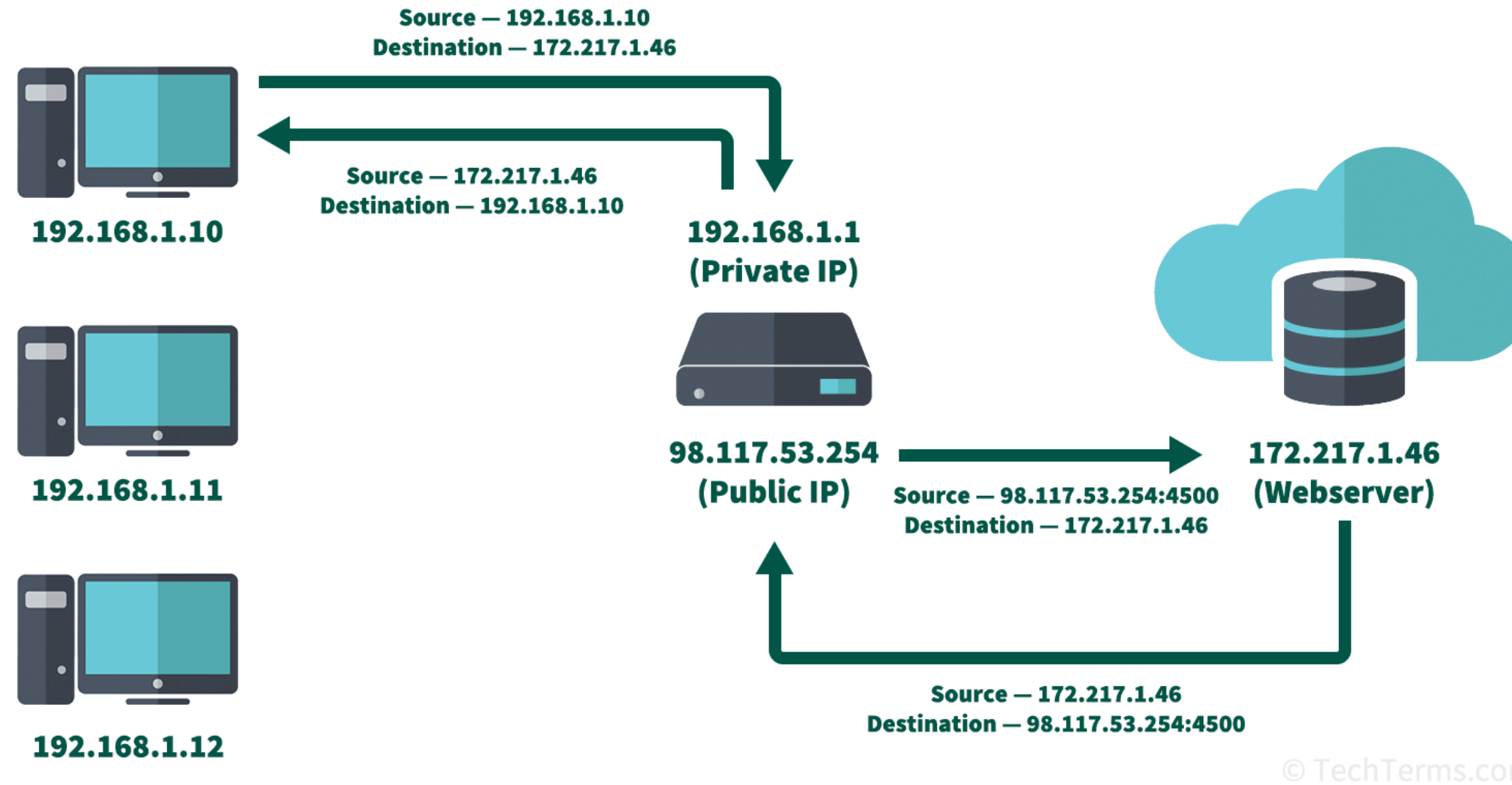
When a response comes back:

1. The NAT device uses the port number to identify the correct internal device.
2. It forwards the response to the appropriate internal IP address.

NAT (Network Address Translation)



NAT (Network Address Translation)



This way, multiple devices can share a single public IP address while maintaining separate connections.

Layer 4: Application



Layer 4: Application Layer

The **Application Layer** contains protocols that applications use directly:

- **HTTP/HTTPS** (Web Browsing)
 - HTTP: Hypertext Transfer Protocol (TCP port 80)
 - HTTPS: HTTP Secure with TLS/SSL encryption (TCP port 443)
 - Used for web pages, APIs, and web services

Layer 4: Application Layer

The **Application Layer** contains protocols that applications use directly:

- **HTTP/HTTPS** (Web Browsing)
 - HTTP: Hypertext Transfer Protocol (TCP port 80)
 - HTTPS: HTTP Secure with TLS/SSL encryption (TCP port 443)
 - Used for web pages, APIs, and web services
- **DNS** (Domain Name System)
 - Translates human-readable domain names to IP addresses
 - Reason why we do not need to remember IPs of websites!
 - Example: `www.google.com` → `142.250.185.46`
 - UDP Port 53

DNS: How Domain Names are Resolved

DNS is a **hierarchical, distributed** database. No single server knows all domain names – instead, the work is split across a tree of servers:

- **Root DNS Servers** (13 clusters worldwide, labeled A–M)
 - Know where to find the TLD servers
- **TLD (Top-Level Domain) Servers** (e.g., .com, .org, .it, .edu)
 - Know where to find authoritative servers for each domain
- **Authoritative DNS Servers**
 - Hold the actual DNS records for a specific domain (e.g., example.com)
 - Return the final IP address

DNS Resolution: Step by Step

EXAMPLE: Resolving `www.example.com`:

1. **Browser cache:** Already resolved recently? Use cached IP
2. **OS cache:** Operating system checks its local DNS cache
3. **Recursive resolver** (e.g., your ISP or `8.8.8.8`): Checks its cache; if not found, starts querying the hierarchy:
 - Asks a **Root Server**: "What is the IP of `www.example.com`?" → "I don't know, but ask the `.com` TLD server at this address"
 - Asks the **.com TLD Server**: "What is the IP of `www.example.com`?" → "I don't know, but ask the authoritative server for `example.com` at this address"
 - Asks the **Authoritative Server** for `example.com`: "What is the IP of `www.example.com`?" → `93.184.216.34`
4. Recursive resolver **caches** the result and returns it to your computer

DNS Caching and TTL

To avoid repeating the full resolution process every time, DNS responses are **cached** at multiple levels:

- **Browser cache:** Stores recent lookups (minutes)
- **OS cache:** System-level DNS cache
- **Recursive resolver cache:** ISP or public resolver (e.g., 8.8.8.8) caches results

DNS Caching and TTL

To avoid repeating the full resolution process every time, DNS responses are **cached** at multiple levels:

- **Browser cache:** Stores recent lookups (minutes)
- **OS cache:** System-level DNS cache
- **Recursive resolver cache:** ISP or public resolver (e.g., 8.8.8.8) caches results

Each DNS record has a **TTL (Time To Live)** – the number of seconds the result can be cached before it must be re-queried.

- Short TTL (e.g., 60s): Faster updates, but more DNS traffic
- Long TTL (e.g., 86400s = 24h): Less traffic, but changes propagate slowly

HTTP: HyperText Transfer Protocol

HTTP is the protocol used by web browsers to communicate with web servers. It follows a simple **request-response** model:

1. Client (browser) sends a **request**: method, URL, headers, optional body
2. Server processes the request and sends a **response**: status code, headers, body

HTTP: HyperText Transfer Protocol

HTTP is the protocol used by web browsers to communicate with web servers. It follows a simple **request-response** model:

- 1. Client (browser) sends a **request**: method, URL, headers, optional body
- 2. Server processes the request and sends a **response**: status code, headers, body

Common HTTP methods:

Method	Purpose	Example
GET	Retrieve a resource	Load a web page
POST	Send data to the server	Submit a form, upload a file
PUT	Update/replace a resource	Update a user profile
DELETE	Remove a resource	Delete an account



HTTP vs HTTPS

HTTP transmits data in **plain text** – anyone intercepting the traffic can read it!

HTTP vs HTTPS

HTTP transmits data in **plain text** – anyone intercepting the traffic can read it!

HTTPS = HTTP + **TLS/SSL** encryption:

- All data between browser and server is **encrypted**
- The server proves its identity with a **certificate**
- Protects against eavesdropping, tampering, and impersonation
- Uses TCP port **443** (instead of port 80 for HTTP)

HTTP vs HTTPS

HTTP transmits data in **plain text** – anyone intercepting the traffic can read it!

HTTPS = HTTP + **TLS/SSL** encryption:

- All data between browser and server is **encrypted**
- The server proves its identity with a **certificate**
- Protects against eavesdropping, tampering, and impersonation
- Uses TCP port **443** (instead of port 80 for HTTP)

NOTE: We will cover HTTP and HTTPS in detail in a dedicated set of slides. For now, remember: **always use HTTPS** – HTTP is insecure!

Layer 4: Application Layer (cont.)

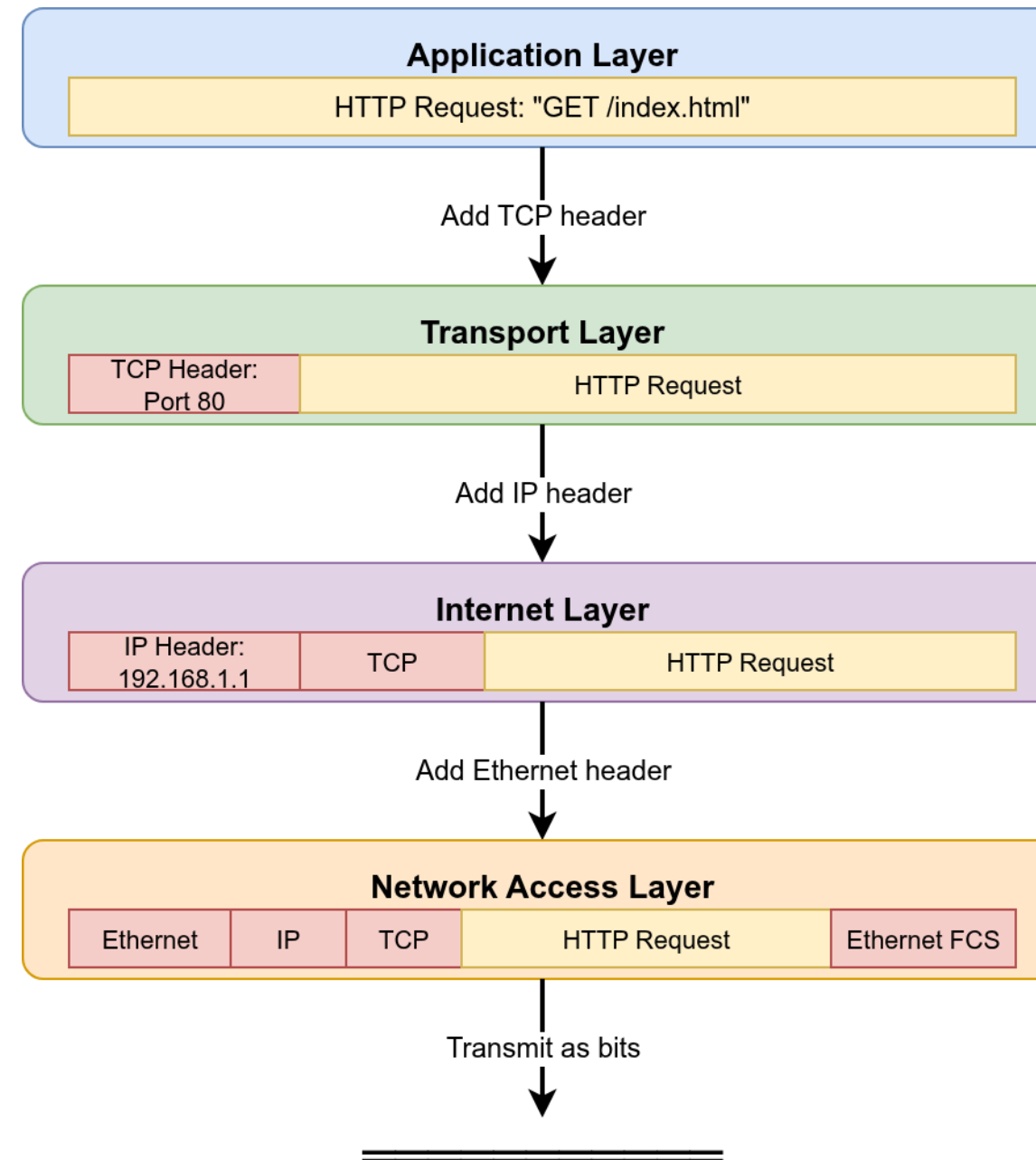
- **SMTP/POP3/IMAP** (Email when using an email client)
 - SMTP: Sending emails (TCP port 25, 587)
 - POP3: Retrieving emails, downloads and deletes (TCP port 110)
 - IMAP: Retrieving emails, keeps on server (TCP port 143)

Layer 4: Application Layer (cont.)

- **SMTP/POP3/IMAP** (Email when using an email client)
 - SMTP: Sending emails (TCP port 25, 587)
 - POP3: Retrieving emails, downloads and deletes (TCP port 110)
 - IMAP: Retrieving emails, keeps on server (TCP port 143)
- **FTP/SFTP** (File Transfer)
 - FTP: File Transfer Protocol (TCP ports 20, 21)
 - SFTP: Secure FTP over SSH (TCP port 22)

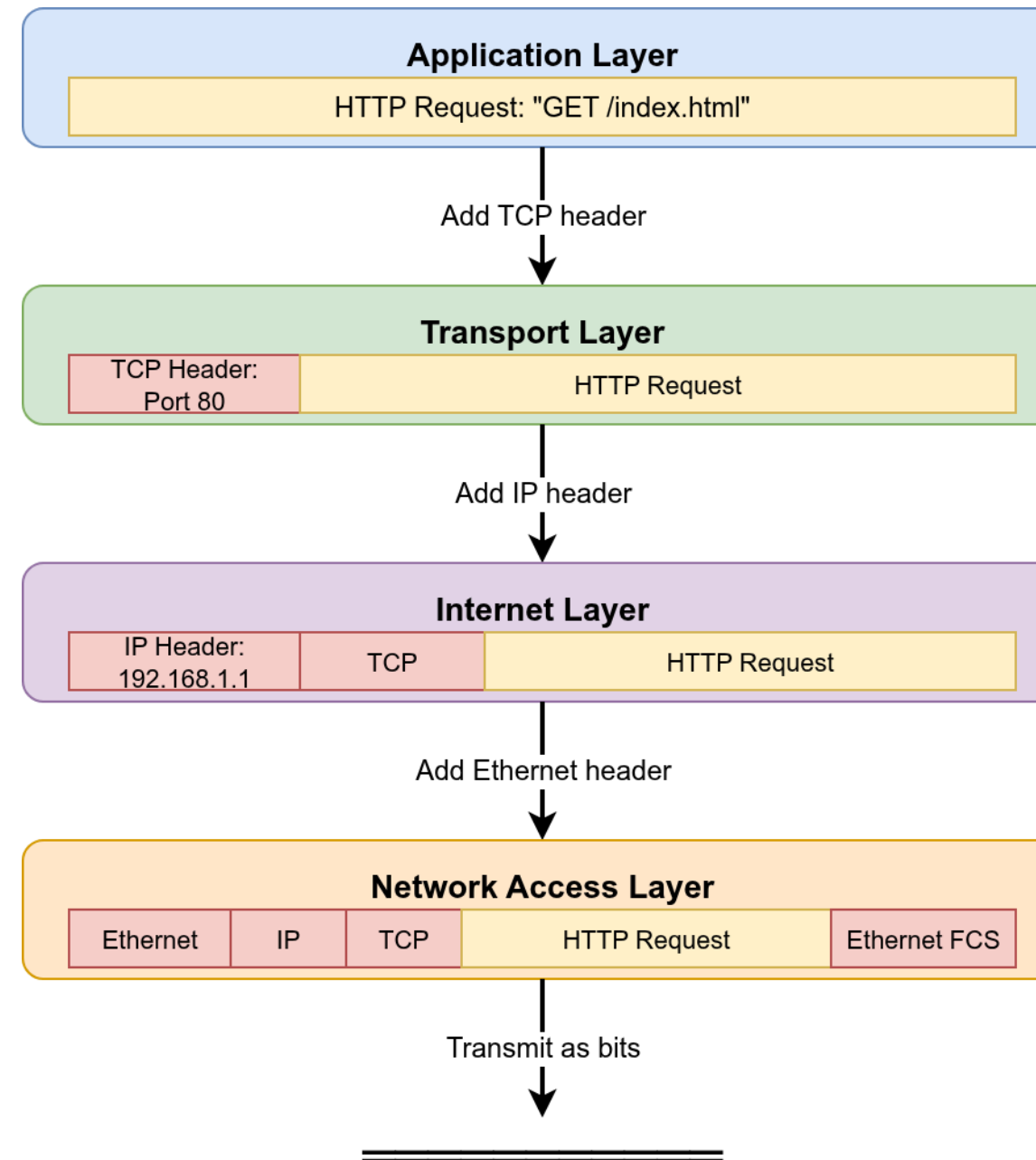
How Data Flows Through the Stack

When you send data, each layer adds its own **header** (encapsulation):



How Data Flows Through the Stack

When you send data, each layer adds its own **header** (encapsulation):



On the receiving side, each layer **removes** its header (decapsulation) and passes data up.

Example: Loading a Web Page



Example: Loading a Web Page

Let's trace what happens when you visit `https://www.example.com`:

1. DNS Resolution (finding the IP address):

- **Your computer:**

1. The user types `www.example.com` in the browser
2. Browser checks if IP is cached locally; if not, sends DNS query
3. **Application Layer:** DNS query "What is the IP address of `www.example.com`?"
4. **Transport Layer:** Wraps query in UDP packet (UDP is fast, suitable for DNS)
 - Source Port: random high-numbered port (e.g., 54321)
 - Destination Port: 53 (DNS)
5. **Internet Layer:** Adds IP header
 - Source IP: your computer's private IP (e.g., 192.168.1.10)
 - Destination IP: DNS server's IP (e.g., 8.8.8.8 - Google's DNS)
6. **Network Access Layer:** Wraps in Ethernet/WiFi frame and sends to your router (default gateway)

Example: Loading a Web Page (DNS - Router)

- **Your router:**

1. Receives the frame, extracts the UDP/IP packet
2. **NAT:** Replaces your private IP with router's public IP and tracks the mapping
 - Source IP: router's public IP (e.g., 203.0.113.45)
 - Source Port: new port number for tracking (e.g., 12345)
3. Forwards the packet to the internet

Example: Loading a Web Page (DNS - Server)

- **The Internet:**

1. Multiple routers forward the packet hop-by-hop based on destination IP (8.8.8.8)
2. Each router examines the IP header and forwards to the next router

- **DNS Server** (e.g., 8.8.8.8):

1. Receives the UDP/IP packet on port 53
2. Extracts the DNS query from the UDP payload
3. Looks up `www.example.com` in its database
4. Finds the IP address: `93.184.216.34`
5. Builds DNS response packet with the IP address
6. Sends UDP/IP packet back:
 - Source IP: 8.8.8.8, Source Port: 53
 - Destination IP: 203.0.113.45 (your router's public IP)
 - Destination Port: 12345 (the port NAT assigned)

Example: Loading a Web Page (DNS - Response)

- **The Internet:**

1. Routers forward the response packet back to your router

- **Your router:**

1. Receives the packet on port 12345
2. **NAT:** Looks up the port mapping and translates back
 - Destination IP: 192.168.1.10 (your computer's private IP)
 - Destination Port: 54321 (original port)
3. Forwards packet to your computer on the local network

- **Your computer:**

1. Receives the UDP/IP packet on port 54321
2. Extracts the DNS response: `www.example.com = 93.184.216.34`
3. Browser now knows the server's IP address!

Example: Loading a Web Page (HTTPS - Connection)

2. HTTPS Connection and Request:

- **Your computer:**

1. Browser initiates connection to 93.184.216.34 on port 443 (HTTPS)
2. **Transport Layer - TCP 3-way handshake** (establishing reliable connection):
 - Your computer → Server: SYN (synchronize)
 - Server → Your computer: SYN-ACK (synchronize-acknowledge)
 - Your computer → Server: ACK (acknowledge)
3. **Application Layer - TLS/SSL handshake** (establishing secure connection):
 - Negotiate encryption algorithms
 - Exchange certificates
 - Generate session keys
4. **Application Layer:** Browser builds HTTPS GET request
 - GET / HTTP/1.1
 - Host: www.example.com

Example: Loading a Web Page (HTTPS - Sending)

- **Your computer:**

- 5. **Transport Layer:** Wraps in TCP segment

- Source Port: random high-numbered port (e.g., 54322)
 - Destination Port: 443 (HTTPS)

- 6. **Internet Layer:** Adds IP header

- Source IP: 192.168.1.10 (your private IP)
 - Destination IP: 93.184.216.34

- 7. **Network Access Layer:** Sends to router

- **Your router:**

- 1. Receives the frame, extracts TCP/IP packet

- 2. **NAT:** Translates addresses

- Source IP: 203.0.113.45 (router's public IP)
 - Source Port: 23456 (new tracking port)

- 3. Forwards to the internet

Example: Loading a Web Page (HTTPS - Server)

- **The Internet:**

1. Routers forward the packet hop-by-hop to destination IP (93.184.216.34)

- **Web Server (93.184.216.34):**

1. Receives TCP/IP packet on port 443
2. Decrypts the HTTPS request using TLS session keys
3. Processes the HTTP GET request
4. Retrieves the web page content (HTML, CSS, JavaScript, images)
5. Builds HTTPS response with the content
6. Encrypts the response
7. Sends TCP/IP packet back:
 - Source IP: 93.184.216.34, Source Port: 443
 - Destination IP: 203.0.113.45 (your router's public IP)
 - Destination Port: 23456 (NAT tracking port)

Example: Loading a Web Page (HTTPS - Result)

- **The Internet:**

1. Routers forward the response back to your router

- **Your router:**

1. Receives the packet on port 23456
2. **NAT:** Translates back using port mapping
 - Destination IP: 192.168.1.10 (your computer)
 - Destination Port: 54322 (original port)
3. Forwards to your computer

- **Your computer:**

1. Receives TCP/IP packet on port 54322
2. **Transport Layer:** TCP ensures data arrived correctly and in order
3. **Application Layer:** Browser decrypts HTTPS response
4. Browser parses HTML, loads CSS/JavaScript, renders images
5. **You see the web page!**

Example: Loading a Web Page (HTTPS - Result)

- **The Internet:**

1. Routers forward the response back to your router

- **Your router:**

1. Receives the packet on port 23456
2. **NAT:** Translates back using port mapping
 - Destination IP: 192.168.1.10 (your computer)
 - Destination Port: 54322 (original port)
3. Forwards to your computer

- **Your computer:**

1. Receives TCP/IP packet on port 54322
2. **Transport Layer:** TCP ensures data arrived correctly and in order
3. **Application Layer:** Browser decrypts HTTPS response
4. Browser parses HTML, loads CSS/JavaScript, renders images
5. **You see the web page!**

Note: This is simplified. In reality, modern web pages require dozens or hundreds of additional requests for resources (images, scripts, stylesheets, fonts, etc.), each following a similar process.

Demo with WireShark



VPN



VPN: Virtual Private Network

A **VPN** creates an **encrypted tunnel** between your device and a VPN server, hiding your traffic from anyone in between.

- All your internet traffic is routed through the VPN server
- Your real IP address is **hidden** – websites see the VPN server's IP instead
- Data is **encrypted** between your device and the VPN server, even on untrusted networks (e.g., public WiFi)

Without VPN: Your Device → Router → ISP → Internet → Website

- ISP can see **all** your traffic (which sites you visit, DNS queries, etc.)
- The website sees **your real IP address**
- On public WiFi, anyone nearby can potentially intercept your traffic

Without VPN: Your Device → Router → ISP → Internet → Website

- ISP can see **all** your traffic (which sites you visit, DNS queries, etc.)
- The website sees **your real IP address**
- On public WiFi, anyone nearby can potentially intercept your traffic

With VPN: Your Device → **encrypted tunnel** → VPN Server → Internet → Website

- ISP only sees **encrypted traffic** to the VPN server (not what you're doing)
- The website sees the **VPN server's IP**, not yours
- Traffic is protected even on untrusted networks

Without VPN: Your Device → Router → ISP → Internet → Website

- ISP can see **all** your traffic (which sites you visit, DNS queries, etc.)
- The website sees **your real IP address**
- On public WiFi, anyone nearby can potentially intercept your traffic

With VPN: Your Device → **encrypted tunnel** → VPN Server → Internet → Website

- ISP only sees **encrypted traffic** to the VPN server (not what you're doing)
- The website sees the **VPN server's IP**, not yours
- Traffic is protected even on untrusted networks

WARNING: The VPN may track you and can see your encrypted traffic!

VPN Use Case 1: Remote Work

A company has internal services (file servers, databases, internal web apps) accessible only from the **office LAN**. Employees working from home are on a **different network** and cannot reach those services directly.

VPN Use Case 1: Remote Work

A company has internal services (file servers, databases, internal web apps) accessible only from the **office LAN**. Employees working from home are on a **different network** and cannot reach those services directly.

With a **corporate VPN**:

1. The employee connects to the company's **VPN server** over the internet
2. An encrypted tunnel is established between the employee's device and the VPN server
3. The employee's device gets a **virtual IP address** on the company's internal network
4. Now the employee can access internal services **as if they were in the office**

The company controls the VPN server and can enforce access policies, logging, and authentication.

VPN Use Case 2: Consumer VPN (Privacy)

Ordinary users can subscribe to a **consumer VPN service** (e.g., NordVPN, Mullvad, ProtonVPN) to protect their privacy online.

VPN Use Case 2: Consumer VPN (Privacy)

Ordinary users can subscribe to a **consumer VPN service** (e.g., NordVPN, Mullvad, ProtonVPN) to protect their privacy online.

Goals:

- **Hide browsing activity from the ISP:** The ISP only sees encrypted traffic to the VPN server
- **Mask your IP address:** Websites see the VPN server's IP, not yours
- **Bypass geo-restrictions:** Connect through a server in another country to access region-locked content (e.g., streaming services)
- **Protect on public WiFi:** Encrypt all traffic on untrusted networks (airports, cafés, hotels)

VPN Use Case 2: Consumer VPN (Privacy)

Ordinary users can subscribe to a **consumer VPN service** (e.g., NordVPN, Mullvad, ProtonVPN) to protect their privacy online.

Goals:

- **Hide browsing activity from the ISP:** The ISP only sees encrypted traffic to the VPN server
- **Mask your IP address:** Websites see the VPN server's IP, not yours
- **Bypass geo-restrictions:** Connect through a server in another country to access region-locked content (e.g., streaming services)
- **Protect on public WiFi:** Encrypt all traffic on untrusted networks (airports, cafés, hotels)

WARNING: You are **trusting the VPN provider** with all your traffic! A dishonest VPN provider can log, inspect, or sell your browsing data. Choose carefully.

VPN Implementations

Two widely used open-source VPN implementations:

VPN Implementations

Two widely used open-source VPN implementations:

OpenVPN (2001):

- Mature, battle-tested, widely supported
- Uses **OpenSSL** for encryption (TLS-based)
- Runs in **user space** (no kernel modifications needed)
- Supports both **UDP** (default, faster) and **TCP** (port 443, for restrictive networks)
- Large codebase (100,000 lines of C) → harder to audit
- Used extensively in corporate and consumer VPNs

VPN Implementations

Two widely used open-source VPN implementations:

OpenVPN (2001):

- Mature, battle-tested, widely supported
- Uses **OpenSSL** for encryption (TLS-based)
- Runs in **user space** (no kernel modifications needed)
- Supports both **UDP** (default, faster) and **TCP** (port 443, for restrictive networks)
- Large codebase (100,000 lines of C) → harder to audit
- Used extensively in corporate and consumer VPNs

WireGuard (2018):

- Modern, minimalist design
- Uses state-of-the-art cryptography (ChaCha20, Curve25519, BLAKE2s)
- Runs in **kernel space** → very fast, low latency
- **UDP only** by default
- Tiny codebase (4,000 lines of C) → easy to audit and verify
- Built into the Linux kernel since version 5.6
- Increasingly adopted by consumer VPN providers

VPN: UDP vs TCP and Detection

Most VPNs use **UDP** by default for performance (lower overhead, no double reliability layer with TCP-over-TCP).

VPN: UDP vs TCP and Detection

Most VPNs use **UDP** by default for performance (lower overhead, no double reliability layer with TCP-over-TCP).

However, UDP-based VPN traffic is **easy to detect**:

- Firewalls and ISPs can identify unusual patterns of UDP traffic on non-standard ports
- Some countries and corporate networks **block** known VPN protocols entirely

VPN: UDP vs TCP and Detection

Most VPNs use **UDP** by default for performance (lower overhead, no double reliability layer with TCP-over-TCP).

However, UDP-based VPN traffic is **easy to detect**:

- Firewalls and ISPs can identify unusual patterns of UDP traffic on non-standard ports
- Some countries and corporate networks **block** known VPN protocols entirely

To evade detection, some VPN implementations can switch to **TCP port 443** (the same port used by HTTPS):

- VPN traffic looks like **normal HTTPS traffic** to the network
- Much harder for firewalls and ISPs to distinguish from regular web browsing
- Trade-off: **lower performance** due to TCP overhead (and potential TCP-over-TCP issues)

OpenVPN natively supports TCP mode. WireGuard (UDP-only) can be wrapped in TCP using tools like `udp2raw` or `wstunnel`.